

09 - Notifications, Settings, Privacy, Offline Operations, and Release Hardening

Feature objective

Finish the mobile product as a trustworthy Android application: meaningful notifications, transparent SMS/privacy controls, selective offline support, secure data handling, and explicit Play Store release gates.

Dependencies

All core domain features.

In scope

- in-app notifications
- Expo push notifications
- deep linking
- notification preferences
- profile/settings
- SMS settings/status
- privacy/data controls
- session/security UI
- offline banner/outbox status
- deletion flows
- Play Store SMS permission readiness
- accessibility/performance/release checklist

Out of scope

- browser push
- advanced analytics
- Sentry/PostHog in MVP
- biometric lock unless added later
- iOS release

Routes

```
notifications.tsx
profile.tsx
settings/index.tsx
settings/sms.tsx
settings/notifications.tsx
settings/privacy.tsx
settings/security.tsx
settings/data.tsx
```

Notification philosophy

Only notify for financially meaningful events.

Three levels:

- Info
- Important
- Action required

Examples:

Action required

- plan requires review
- transaction conflict requires confirmation
- critical stale financial data blocks trustworthy planning

Important

- EMI due soon
- goal materially falling behind
- unusual transaction

Info

- monthly review ready
- AI research completed
- plan generation completed

Do not use engagement spam.

Push content privacy

Do not put sensitive financial values in lock-screen push text by default.

Good:

Your monthly financial review is ready.

Riskier:

Your SBI balance is INR 18,331 and you spent INR 31,400.

Detailed values should appear after the user opens the app.

Deep links

Notifications should route directly to:

- transaction
- goal
- plan drift review
- loan
- AI job/conversation

Route contract must remain stable.

Notifications screen

Group by urgency/date.

Each row:

- icon/type
- title
- short context
- time
- read/unread

No giant notification cards unless action is required.

SMS settings

Show clearly:

- permission status

- last scan
- last successful sync
- parser version if useful for support
- rolling scan behavior

Actions:

- Sync now
- Review transactions
- Clear local SMS-derived cache
- Open Android permission settings

Disclosure:

- raw personal SMS is not uploaded
- financial parsing happens locally before normalized records sync

Privacy and Data

Provide explicit sections:

- What data we store
- What stays on device
- Download/export my data when backend supports it
- Delete financial data
- Delete account
- Clear local SMS-derived cache

Separate local cache deletion from backend account deletion.

Security UI

MVP:

- active session/device list if backend exposes it
- revoke session
- logout
- re-authentication for destructive account/security changes when backend requires it

Future:

- biometric app lock

Offline banner

Do not block the app with a full-screen offline modal.

Use a compact banner/status:

Offline - showing saved data

Pending writes can show:

2 changes waiting to sync

Outbox

Outbox operations need:

- local unique ID
- operation type
- payload
- created time
- retry count
- idempotency key

- last error

Sync policy:

- trigger when app becomes online/foreground
- exponential/backoff-ish retries rather than tight loops
- stop retrying permanent validation errors
- expose failed item to user when intervention is required

Error-state policy

Failure	UX
Offline	cached data + banner
API down	preserve last-known data + Retry
SMS denied	manual transaction mode continues
SMS blocked	explain and open settings
Partial sync fail	keep successful items; retry failed
AI fail	retry without deleting chat
Plan generation fail	current plan remains active
Stale account	freshness warning
Auth refresh fail	clear session and login

Accessibility release rules

- minimum approximately 48dp targets
- text scaling works
- icon-only buttons have labels
- status does not rely only on color
- reduced motion respected
- charts have textual summaries
- focus/announcement after important async completion where practical
- error text is explicit

Performance release rules

- FlashList for large transaction lists
- avoid rerendering entire tab on one transaction mutation
- render cached Query data before skeleton when available
- defer expensive charts until visible
- avoid giant JSON context passing to AI from mobile
- keep raw SMS processing incremental after first scan
- measure Android cold start on a mid-range device, not only emulator

Play Store SMS release gate

Because SMS access is restricted, treat policy work as a formal release milestone.

Before production submission:

- [] SMS use is genuinely core functionality
- [] store listing clearly explains SMS-based money management
- [] only necessary SMS permission is requested
- [] permission request happens in context after rationale
- [] Permissions Declaration Form prepared/submitted
- [] privacy policy is public and accurate
- [] raw personal/non-financial SMS is not uploaded
- [] local filtering behavior can be demonstrated
- [] manual transaction fallback works without SMS permission

- [] user can revoke permission and app remains usable
- [] sensitive SMS/financial values are not logged
- [] account/data deletion path works

Validate Google Play policy again immediately before release because policy can change.

Build/release workflow

Development:

`expo run:android` for native SMS module work

Expo Development Build for normal testing

Preview/internal:

EAS Build -> internal testers

Production:

EAS Build -> Play Console testing track -> production

Release progression:

1. developer emulator/device
2. internal build
3. small closed testing group
4. Play policy validation
5. wider closed/open testing if useful
6. production

Step-by-step implementation

1. Build Notifications list.
2. Add Expo push token registration.
3. Add deep-link handler.
4. Build notification preferences.
5. Build Settings index/profile.
6. Build SMS settings/status.
7. Build Privacy & Data pages.
8. Add session revoke/security page.
9. Add offline/outbox status UI.
10. Add destructive-action confirmation/re-auth flows.
11. Perform accessibility pass.
12. Perform performance pass.
13. Prepare store disclosure/privacy text.
14. Run closed-test release checklist.

Acceptance criteria

- push/in-app notification opens correct destination
- sensitive values are absent from default push text
- SMS permission can be revoked without breaking app
- user can understand what SMS data leaves device
- offline cached screens remain usable
- pending outbox state is visible when relevant
- deletion flows are explicit and distinct
- app passes internal accessibility checklist
- release candidate contains no raw SMS logging

Tests

- deep-link route tests

- notification rendering/priority
- offline banner/outbox states
- permission revoked state
- privacy destructive flows
- session revoke flow
- push payload privacy review